ROLASOFT TECHNOLOGIES LIMITED

RolaSoft Technologies Limited is a software company. We are also into Professional Computer Education, International School Admission Processes, Social Media Marketing.

Website: www.rolasofttech.com

Email: info@rolasofttech.com

JavaScript Textbook

This tutorial has been prepared for JavaScript beginners to help them understand the basic functionality of JavaScript to build dynamic web pages and web applications.

Prerequisites

For this tutorial, it is assumed that the reader have a prior knowledge of HTML coding. It would help if the reader had some prior exposure to object-oriented programming concepts and a general idea on creating online applications.

tutorialspoint
SIMPLYEASYLEARNING

Table of Contents

About the Tutorial i
Audience i
Prerequisites i
Copyright and Disclaimer i
Table of Contents ii

PART 1: JAVASCRIPT BASICS 1

1. **Overview** ..... **2** What is JavaScript? 2
Client-Side JavaScript 2
Advantages of JavaScript 3
Limitations of JavaScript 3
JavaScript Development Tools 3
Where is JavaScript Today? 4

2. **Syntax** ..... **5** Your First JavaScript Code 5
Whitespace and Line Breaks 6
Semicolons are Optional 6
Case Sensitivity 7
Comments in JavaScript 7

3. **Enabling** ..... **9** JavaScript in Internet Explorer 9
JavaScript in Firefox 9
JavaScript in Chrome 10
JavaScript in Opera 10
Warning for Non-JavaScript Browsers 10

4. **Placement** ..... **12** JavaScript in <head>...</head> Section 12
JavaScript in <body>...</body> Section 13
JavaScript in <body> and <head> Sections 13
JavaScript in External File 14

tutorialspoint
SIMPLYEASYLEARNING

5. **Variables** ..... **16** JavaScript Datatypes..... 16
JavaScript Variables 16
JavaScript Variable Scope 17
JavaScript Variable Names 18
JavaScript Reserved Words 19

6. **Operators**..... **20** What is an Operator? 20
Arithmetic Operators..... 20
Comparison Operators 23
Logical Operators 26
Assignment Operators 31
Miscellaneous Operators 34

7. **If-Else** ..... **38** Flow Chart of if-else ..... 38 if Statement ..... 39 if...else Statement 40
if...else if... Statement 41

8. **Switch-Case**..... **43**
Flow Chart 43

9. **While Loop** ..... **47** The while Loop 47
The do...while Loop 49

10. **For Loop** **52**
The for Loop 52

11. **For-in Loop** **55**

12. **Loop Control** ..... **57** The break Statement 57
The continue Statement 59
Using Labels to Control the Flow 60

tutorialspoint
SIMPLYEASYLEARNING

13. Functions .....
64 Function Definition
..... 64
Calling a Function 65
Function Parameters 66
The return Statement 67
Nested Functions .....
68
Function () Constructor 70
Function Literals 71

14. Events .....
74 What is an Event?
..... 74 onclick Event
Type ..... 74 onsubmit
Event Type 75
onmouseover and onmouseout .....
76
HTML 5 Standard Events 77

15. Cookies .....
82 What are
Cookies?..... 82
How It Works? 82
Storing Cookies 83
Reading Cookies 84
Setting Cookies Expiry Date 86
Deleting a Cookie 87

tutorialspoint
SIMPLYEASYLEARNING

16. Page Redirect **89**
What is Page Redirection? 89
JavaScript Page Refresh 89
Auto Refresh 89
How Page Re-direction Works? 90

17. Dialog Box **94**
Alert Dialog Box 94
Confirmation Dialog Box 95
Prompt Dialog Box 96

18. Void Keyword **98**

19. Page Printing **101**
How to Print a Page? 102

tutorialspoint
SIMPLYEASYLEARNING

1. OVERVIEW

Part 1: JavaScript Basics

What is JavaScript?

Javascript is a dynamic computer programming language. It is lightweight and most commonly used as a part of web pages, whose implementations allow client-side script to interact with the user and make dynamic pages. It is an interpreted programming language with object-oriented capabilities.

JavaScript was first known as **LiveScript**, but Netscape changed its name to JavaScript, possibly because of the excitement being generated by Java. JavaScript made its first appearance in Netscape 2.0 in 1995 with the name **LiveScript**. The general-purpose core of the language has been embedded in Netscape, Internet Explorer, and other web browsers.

The ECMA-262 Specification defined a standard version of the core JavaScript language.

- JavaScript is a lightweight, interpreted programming language.
- Designed for creating network-centric applications.
- Complementary to and integrated with Java.
- Complementary to and integrated with HTML.
- Open and cross-platform.

Client-Side JavaScript

Client-side JavaScript is the most common form of the language. The script should be included in or referenced by an HTML document for the code to be interpreted by the browser.

It means that a web page need not be a static HTML, but can include programs that interact with the user, control the browser, and dynamically create HTML content.

The JavaScript client-side mechanism provides many advantages over traditional CGI server-side scripts. For example, you might use JavaScript to check if the user has entered a valid e-mail address in a form field.

tutorialspoint
SIMPLYEASYLEARNING

The JavaScript code is executed when the user submits the form, and only if all the entries are valid, they would be submitted to the Web Server.

JavaScript can be used to trap user-initiated events such as button clicks, link navigation, and other actions that the user initiates explicitly or implicitly.

Advantages of JavaScript

The merits of using JavaScript are:

- **Less server interaction:** You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- **Immediate feedback to the visitors:** They don't have to wait for a page reload to see if they have forgotten to enter something.
- **Increased interactivity:** You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- **Richer interfaces:** You can use JavaScript to include such items as dragand-drop components and sliders to give a Rich Interface to your site visitors.

Limitations of JavaScript

We cannot treat JavaScript as a full-fledged programming language. It lacks the following important features:

- Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reason.
- JavaScript cannot be used for networking applications because there is no such support available.
- JavaScript doesn't have any multithreading or multiprocessor capabilities.

Once again, JavaScript is a lightweight, interpreted programming language that allows you to build interactivity into otherwise static HTML pages.

JavaScript Development Tools

One of major strengths of JavaScript is that it does not require expensive development tools. You can start with a simple text editor such as Notepad. Since it is an interpreted language inside the context of a web browser, you don't even need to buy a compiler.

To make our life simpler, various vendors have come up with very nice JavaScript editing tools. Some of them are listed here:

- **Microsoft FrontPage:** Microsoft has developed a popular HTML editor called FrontPage. FrontPage also provides web developers with a number of JavaScript tools to assist in the creation of interactive websites.
- **Macromedia Dreamweaver MX:** Macromedia Dreamweaver MX is a very popular HTML and JavaScript editor in the professional web development crowd. It provides several handy prebuilt JavaScript

components, integrates well with databases, and conforms to new standards such as XHTML and XML.

- **Macromedia HomeSite 5:** HomeSite 5 is a well-liked HTML and JavaScript editor from Macromedia that can be used to manage personal websites effectively.

Where is JavaScript Today?

The ECMAScript Edition 5 standard will be the first update to be released in over four years. JavaScript 2.0 conforms to Edition 5 of the ECMAScript standard, and the difference between the two is extremely minor.

The specification for JavaScript 2.0 can be found on the following site: http://www.ecmascript.org/

Today, Netscape's JavaScript and Microsoft's JScript conform to the ECMAScript standard, although both the languages still support the features that are not a part of the standard.

tutorialspoint
SIMPLYEASYLEARNING

2. SYNTAX

JavaScript can be implemented using JavaScript statements that are placed within the **<script>... </script>** HTML tags in a web page.

You can place the **<script>** tags, containing your JavaScript, anywhere within you web page, but it is normally recommended that you should keep it within the **<head>** tags.

The <script> tag alerts the browser program to start interpreting all the text between these tags as a script. A simple syntax of your JavaScript will appear as follows.

```
<script ...>
  JavaScript code
</script>
```

The script tag takes two important attributes:

- **Language:** This attribute specifies what scripting language you are using. Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
- **Type:** This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".

So your JavaScript syntax will look as follows.

```
<script language="javascript" type="text/javascript">
  JavaScript code
</script>
```

Your First JavaScript Code

Let us take a sample example to print out "Hello World". We added an optional HTML comment that surrounds our JavaScript code. This is to save our code from a browser that does not support JavaScript. The comment ends with a "//->". Here "//" signifies a comment in JavaScript, so we add that to prevent a browser from reading the end of the HTML comment as a piece of JavaScript code. Next, we call a function **document.write** which writes a string into our HTML document.

This function can be used to write text, HTML, or both. Take a look at the following code.

```
<html>
<body>
<script language="javascript" type="text/javascript">
<!--
   document.write ("Hello World!")
//-->
</script>
</body> </html>
```

This code will produce the following result:

```
Hello World!
```

Whitespace and Line Breaks

JavaScript ignores spaces, tabs, and newlines that appear in JavaScript programs. You can use spaces, tabs, and newlines freely in your program and you are free to format and indent your programs in a neat and consistent way that makes the code easy to read and understand.

Semicolons are Optional

Simple statements in JavaScript are generally followed by a semicolon character, just as they are in C, C++, and Java. JavaScript, however, allows you to omit this semicolon if each of your statements are placed on a separate line. For example, the following code could be written without semicolons.

```
<script language="javascript" type="text/javascript">
<!--
var1 = 10
var2 = 20
//-->
</script>
```


tutorialspoint
SIMPLYEASYLEARNING

tutorialspoint
SIMPLYEASYLEARNING

But when formatted in a single line as follows, you must use semicolons:

```
<script language="javascript" type="text/javascript">
<!--
  var1 = 10; var2 = 20;
//-->
</script>
```

Note: It is a good programming practice to use semicolons.

Case Sensitivity

JavaScript is a case-sensitive language. This means that the language keywords, variables, function names, and any other identifiers must always be typed with a consistent capitalization of letters.

So the identifiers **Time** and **TIME** will convey different meanings in JavaScript.

NOTE: Care should be taken while writing variable and function names in JavaScript.

Comments in JavaScript

JavaScript supports both C-style and C++-style comments. Thus:

- Any text between a // and the end of a line is treated as a comment and is ignored by JavaScript.
- Any text between the characters /* and */ is treated as a comment. This may span multiple lines.
- JavaScript also recognizes the HTML comment opening sequence <!--. JavaScript treats this as a single-line comment, just as it does the // comment.
- The HTML comment closing sequence --> is not recognized by JavaScript so it should be written as //-->.

Example

The following example shows how to use comments in JavaScript.

```
<script language="javascript" type="text/javascript">
<!--

// This is a comment. It is similar to comments in C++

/*
* This is a multiline comment in JavaScript
* It is very similar to comments in C Programming
 */
//-->
</script>
```


tutorialspoint
SIMPLYEASYLEARNING

3. ENABLING

All the modern browsers come with built-in support for JavaScript. Frequently, you may need to enable or disable this support manually. This chapter explains the procedure of enabling and disabling JavaScript support in your browsers: Internet Explorer, Firefox, chrome, and Opera.

JavaScript in Internet Explorer

Here are the steps to turn on or turn off JavaScript in Internet Explorer:

- Follow **Tools -> Internet Options** from the menu.
- Select **Security** tab from the dialog box.
- Click the **Custom Level** button.
- Scroll down till you find the **Scripting** option.
- Select *Enable* radio button under **Active scripting**.
- Finally click OK and come out.

To disable JavaScript support in your Internet Explorer, you need to select **Disable** radio button under **Active scripting**.

JavaScript in Firefox

Here are the steps to turn on or turn off JavaScript in Firefox:

- Open a new tab -> type **about: config** in the address bar.
- Then you will find the warning dialog. Select **I'll be careful, I promise!**
- Then you will find the list of **configure options** in the browser.
- In the search bar, type **javascript.enabled**.
- There you will find the option to enable or disable javascript by rightclicking on the value of that option -> **select toggle**.

If javascript.enabled is true; it converts to false upon clicking **toogle**. If javascript is disabled; it gets enabled upon clicking toggle.

JavaScript in Chrome

Here are the steps to turn on or turn off JavaScript in Chrome:

- Click the Chrome menu at the top right hand corner of your browser.
- Select **Settings**.
- Click **Show advanced settings** at the end of the page.
- Under the **Privacy** section, click the Content settings button.
- In the "Javascript" section, select "Do not allow any site to run JavaScript" or "Allow all sites to run JavaScript (recommended)".

JavaScript in Opera

Here are the steps to turn on or turn off JavaScript in Opera:

- Follow **Tools-> Preferences** from the menu.
- Select **Advanced** option from the dialog box.
- Select **Content** from the listed items.
- Select **Enable JavaScript** checkbox.
- Finally click OK and come out.

To disable JavaScript support in Opera, you should not select the **Enable JavaScript checkbox**.

Warning for Non-JavaScript Browsers

If you have to do something important using JavaScript, then you can display a warning message to the user using **<noscript>** tags.

You can add a **noscript** block immediately after the script block as follows:

```
<html>
<body>

<script language="javascript" type="text/javascript">
<!--
   document.write ("Hello World!")
//-->
```

```
</script>
```


tutorialspoint
SIMPLYEASYLEARNING

```
<noscript>
  Sorry...JavaScript is needed to go ahead.
</noscript>
</body>
</html>
```

Now, if the user's browser does not support JavaScript or JavaScript is not enabled, then the message from </noscript> will be displayed on the screen.

tutorialspoint
SIMPLYEASYLEARNING

4. PLACEMENT

There is a flexibility given to include JavaScript code anywhere in an HTML document. However the most preferred ways to include JavaScript in an HTML file are as follows:

- Script in <head>...</head> section.
- Script in <body>...</body> section.
- Script in <body>...</body> and <head>...</head> sections.
- Script in an external file and then include in <head>...</head> section.

In the following section, we will see how we can place JavaScript in an HTML file in different ways.

JavaScript in <head>...</head> Section

If you want to have a script run on some event, such as when a user clicks somewhere, then you will place that script in the head as follows.

```
<html>
<head>
<script type="text/javascript">
<!--
function sayHello() {
alert("Hello World")
}
//-->
</script>
</head>
<body>
Click here for the result
<input type="button" onclick="sayHello()" value="Say Hello" />
</body> </html>
```

This code will produce the following results:

tutorialspoint
SIMPLYEASYLEARNING

```
Click here for the result
```

Say Hello

JavaScript in <body>...</body> Section

If you need a script to run as the page loads so that the script generates content in the page, then the script goes in the <body> portion of the document. In this case, you would not have any function defined using JavaScript. Take a look at the following code.

```
<html>
<head>
</head>
<body>
<script type="text/javascript">
<!--
document.write("Hello World")
//-->
</script>
<p>This is web page body </p>
</body> </html>
```

This code will produce the following results:

```
Hello World
This is web page body
```

JavaScript in <body> and <head> Sections

You can put your JavaScript code in <head> and <body> section altogether as follows.

```
<html>
<head>
<script type="text/javascript">
```


tutorialspoint
SIMPLYEASYLEARNING

```
<!--
function sayHello() {
alert("Hello World")
}
//-->
</script>
</head>
<body>
<script type="text/javascript">
<!--
document.write("Hello World")
//-->
</script>
<input type="button" onclick="sayHello()" value="Say Hello" />
</body> </html>
```

This code will produce the following result.

HelloWorld

Say Hello

JavaScript in External File

As you begin to work more extensively with JavaScript, you will be likely to find that there are cases where you are reusing identical JavaScript code on multiple pages of a site.

You are not restricted to be maintaining identical code in multiple HTML files. The **script** tag provides a mechanism to allow you to store JavaScript in an external file and then include it into your HTML files.

Here is an example to show how you can include an external JavaScript file in your HTML code using **script** tag and its **src** attribute.

```
<html>
<head>
<script type="text/javascript" src="filename.js" ></script>
</head> <body>
....... </body>
</html>
```

To use JavaScript from an external file source, you need to write all your JavaScript source code in a simple text file with the extension ".js" and then include that file as shown above.

For example, you can keep the following content in **filename.js** file and then you can use **sayHello** function in your HTML file after including the filename.js file.

```
function sayHello() {    alert("Hello World")
}
```

JavaScript Datatypes

One of the most fundamental characteristics of a programming language is the set of data types it supports. These are the type of values that can be represented and manipulated in a programming language.

JavaScript allows you to work with three primitive data types:

- **Numbers**, e.g., 123, 120.50 etc.
- **Strings** of text, e.g. "This text string" etc.
- **Boolean**, e.g. true or false.

JavaScript also defines two trivial data types, **null** and **undefined**, each of which defines only a single value. In addition to these primitive data types, JavaScript supports a composite data type known as **object**. We will cover objects in detail in a separate chapter.

Note: Java does not make a distinction between integer values and floatingpoint values. All numbers in JavaScript are represented as floating-point values.

JavaScript represents numbers using the 64-bit floating-point format defined by the IEEE 754 standard.

JavaScript Variables

Like many other programming languages, JavaScript has variables. Variables can be thought of as named containers. You can place data into these containers and then refer to the data simply by naming the container.

Before you use a variable in a JavaScript program, you must declare it. Variables are declared with the **var** keyword as follows.

```
<script type="text/javascript">
<!-- var
money;
var name;
//-->
</script>
```

You can also declare multiple variables with the same **var** keyword as follows:

```
<script type="text/javascript">
<!-- var money, name;
//-->
</script>
```

Storing a value in a variable is called **variable initialization**. You can do variable initialization at the time of variable creation or at a later point in time when you need that variable.

For instance, you might create a variable named **money** and assign the value 2000.50 to it later. For another variable, you can assign a value at the time of initialization as follows.

```
<script type="text/javascript">
<!--
var name =
"Ali"; var
money; money =
2000.50;
//-->
```


tutorialspoint
SIMPLYEASYLEARNING

```
</script>
```

Note: Use the **var** keyword only for declaration or initialization, once for the life of any variable name in a document. You should not re-declare same variable twice.

JavaScript is **untyped** language. This means that a JavaScript variable can hold a value of any data type. Unlike many other languages, you don't have to tell JavaScript during variable declaration what type of value the variable will hold. The value type of a variable can change during the execution of a program and JavaScript takes care of it automatically.

JavaScript Variable Scope

The scope of a variable is the region of your program in which it is defined. JavaScript variables have only two scopes.

- **Global Variables:** A global variable has global scope which means it can be defined anywhere in your JavaScript code.
- **Local Variables:** A local variable will be visible only within a function where it is defined. Function parameters are always local to that function.

Within the body of a function, a local variable takes precedence over a global variable with the same name. If you declare a local variable or function parameter with the same name as a global variable, you effectively hide the global variable. Take a look into the following example.

```
<script type="text/javascript">
<!--
var myVar = "global"; // Declare a global variable function checkscope( )
{
   var myVar = "local";  // Declare a local variable
document.write(myVar);
```

```
}
//-->
</script>
```

It will produce the following result:

```
Local
```

JavaScript Variable Names

While naming your variables in JavaScript, keep the following rules in mind.

- You should not use any of the JavaScript reserved keywords as a variable name. These keywords are mentioned in the next section. For example, **break** or **boolean** variable names are not valid.
- JavaScript variable names should not start with a numeral (0-9). They must begin with a letter or an underscore character. For example, **123test** is an invalid variable name but **_123test** is a valid one.
- JavaScript variable names are case-sensitive. For example, **Name** and **name** are two different variables.

JavaScript Reserved Words

A list of all the reserved words in JavaScript are given in the following table. They cannot be used as JavaScript variables, functions, methods, loop labels, or any object names.

6. OPERATORS

abstract boolean break byte case catch char class const continue debugger default delete do double	else enum export extends false final finally float for function goto if implements import in	Instanceof int interface long native new null package private protected public return short static super	switch synchronized this throw throws transient true try typeof var void volatile while with

What is an Operator?

Let us take a simple expression **4 + 5 is equal to 9**. Here 4 and 5 are called **operands** and '+' is called the **operator**. JavaScript supports the following types of operators.

- Arithmetic Operators
- Comparison Operators
- Logical (or Relational) Operators
- Assignment Operators
- Conditional (or ternary) Operators

Let’s have a look at all the operators one by one.

Arithmetic Operators

JavaScript supports the following arithmetic operators:

Assume variable A holds 10 and variable B holds 20, then:

S. No.	Operator and Description
1	**+ (Addition)** Adds two operands **Ex:** A + B will give 30
2	**- (Subtraction)** Subtracts the second operand from the first **Ex:** A - B will give -10
3	*** (Multiplication)** Multiply both operands **Ex:** A * B will give 200
4	**/ (Division)**
	Divide the numerator by the denominator **Ex:** B / A will give 2
5	**% (Modulus)** Outputs the remainder of an integer division **Ex:** B % A will give 0
6	**++ (Increment)** Increases an integer value by one **Ex:** A++ will give 11

7	**-- (Decrement)** Decreases an integer value by one **Ex:** A-- will give 9

Note: Addition operator (**+**) works for Numeric as well as Strings. e.g. "a" + 10 will give "a10".

Example

The following code shows how to use arithmetic operators in JavaScript.

```
<html>
<body>

<script type="text/javascript">
<!-- var a = 33; var b =
10; var c = "Test"; var
linebreak = "<br />";

document.write("a + b = "); result
= a + b; document.write(result);
document.write(linebreak);
```


tutorialspoint
SIMPLYEASYLEARNING

tutorialspoint
SIMPLYEASYLEARNING

```
document.write("a - b = "); result
= a - b; document.write(result);
document.write(linebreak);

document.write("a / b = "); result
= a / b; document.write(result);
document.write(linebreak);

document.write("a % b = "); result
= a % b; document.write(result);
document.write(linebreak);

document.write("a + b + c = ");
result = a + b + c;
document.write(result);
document.write(linebreak);
 a =
a++;
document.write("a++ = "); result
= a++; document.write(result);
document.write(linebreak);
 b = b--
;
document.write("b-- = "); result
= b--; document.write(result);
document.write(linebreak);
```


tutorialspoint
SIMPLYEASYLEARNING

```
//-->
</script>

<p>Set the variables to different values and then try...</p>
</body> </html>
```

Output

```
a + b = 43 a - b
= 23 a / b = 3.3
a % b = 3 a + b +
c = 43Test a++ =
33 b-- = 10
Set the variables to different values and then try...
```

Comparison Operators

JavaScript supports the following comparison operators:

Assume variable A holds 10 and variable B holds 20, then:

S.No	Operator and Description
1	**== (Equal)** Checks if the value of two operands are equal or not, if yes, then the condition becomes true. **Ex:** (A == B) is not true.
2	**!= (Not Equal)** Checks if the value of two operands are equal or not, if the values are not equal, then the condition becomes true. **Ex:** (A != B) is true.
3	**> (Greater than)** Checks if the value of the left operand is greater than the value of

tutorialspoint
SIMPLYEASYLEARNING

	the right operand, if yes, then the condition becomes true. **Ex:** (A > B) is not true.
4	**< (Less than)** Checks if the value of the left operand is less than the value of the right operand, if yes, then the condition becomes true. **Ex:** (A < B) is true.
5	**>= (Greater than or Equal to)** Checks if the value of the left operand is greater than or equal to the value of the right operand, if yes, then the condition becomes true. **Ex:** (A >= B) is not true.
6	**<= (Less than or Equal to)** Checks if the value of the left operand is less than or equal to the value of the right operand, if yes, then the condition becomes true. **Ex:** (A <= B) is true.

Example

The following code shows how to use comparison operators in JavaScript.

tutorialspoint
SIMPLYEASYLEARNING

```
<html>
<body>

<script type="text/javascript">
<!-- var a
= 10; var
b = 20;
var linebreak = "<br />";

document.write("(a == b) => ");
result = (a == b);
document.write(result);
document.write(linebreak);
```


tutorialspoint
SIMPLYEASYLEARNING

```
document.write("(a < b) => "); result
= (a < b); document.write(result);
document.write(linebreak);

document.write("(a > b) => "); result
= (a > b); document.write(result);
document.write(linebreak);

document.write("(a != b) => ");
result = (a != b);
document.write(result);
document.write(linebreak);

document.write("(a >= b) => ");
result = (a >= b);
document.write(result);
document.write(linebreak);

document.write("(a <= b) => ");
result = (a <= b);
document.write(result);
document.write(linebreak);

//-->
</script>

<p>Set the variables to different values and different operators and
then try...</p>
</body> </html>
```


tutorialspoint
SIMPLYEASYLEARNING

Output

```
(a == b) => false
(a < b) => true
(a > b) => false
(a != b) => true
(a >= b) => false
(a <= b) => true
Set the variables to different values and different operators and then
try...
```

Logical Operators

JavaScript supports the following logical operators:

Assume variable A holds 10 and variable B holds 20, then:

S.No	Operator and Description
1	**&& (Logical AND)** If both the operands are non-zero, then the condition becomes true. **Ex:** (A && B) is true.
2	**|| (Logical OR)** If any of the two operands are non-zero, then the condition becomes true. **Ex:** (A || B) is true.
3	**! (Logical NOT)** Reverses the logical state of its operand. If a condition is true, then the Logical NOT operator will make it false. **Ex:** ! (A && B) is false.

Example

tutorialspoint
SIMPLYEASYLEARNING

Try the following code to learn how to implement Logical Operators in JavaScript.

```
<html>
<body>

<script type="text/javascript">
<!-- var a =
true; var b =
false;
var linebreak = "<br />";

document.write("(a &&  b) => "); result
= (a && b); document.write(result);
document.write(linebreak);

document.write("(a || b) => ");
result = (a || b);
document.write(result);
document.write(linebreak);

document.write("!(a && b) => "); result
= (!(a && b)); document.write(result);
document.write(linebreak);

//-->
</script>


<p>Set the variables to different values and different operators and
then try...</p>


</body>
</html>
```


tutorialspoint
SIMPLYEASYLEARNING

Output

```
(a && b) => false (a
|| b) => true
!(a && b) => true
Set the variables to different values and different operators and then
try...
```

Assignment Operators

JavaScript supports the following assignment operators:

S.No	Operator and Description
1	**= (Simple Assignment)** Assigns values from the right side operand to the left side operand **Ex:** C = A + B will assign the value of A + B into C
2	**+= (Add and Assignment)** It adds the right operand to the left operand and assigns the result to the left operand. **Ex:** C += A is equivalent to C = C + A
3	**-= (Subtract and Assignment)** It subtracts the right operand from the left operand and assigns the result to the left operand. **Ex:** C -= A is equivalent to C = C - A

tutorialspoint
SIMPLYEASYLEARNING

4	***= (Multiply and Assignment)** It multiplies the right operand with the left operand and assigns the result to the left operand. **Ex:** C *= A is equivalent to C = C * A
5	**/= (Divide and Assignment)** It divides the left operand with the right operand and assigns the result to the left operand.
	Ex: C /= A is equivalent to C = C / A
6	**%= (Modules and Assignment)** It takes modulus using two operands and assigns the result to the left operand. **Ex:** C %= A is equivalent to C = C % A

Note: Same logic applies to Bitwise operators, so they will become <<=, >>=, >>=, &=, |= and ^=.

Example

Try the following code to implement assignment operator in JavaScript.

tutorialspoint
SIMPLYEASYLEARNING

```
<html>
<body>

<script type="text/javascript">
<!-- var a
= 33; var
b = 10;
var linebreak = "<br />";

document.write("Value of a => (a = b) =>
"); result = (a = b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a += b) =>
"); result = (a += b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a -= b) =>
"); result = (a -= b);
document.write(result);
```


tutorialspoint
SIMPLYEASYLEARNING

```
document.write(linebreak);

document.write("Value of a => (a *=  b) =>
"); result = (a *= b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a /= b) =>
"); result = (a /= b);
document.write(result);
document.write(linebreak);

document.write("Value of a => (a %= b) =>
"); result = (a %= b);
document.write(result);
document.write(linebreak);

//-->
</script>

<p>Set the variables to different values and different operators and
then try...</p>
</body> </html>
```

Output

```
Value of a => (a = b) => 10
Value of a => (a += b) => 20
Value of a => (a -= b) => 10
Value of a => (a *= b) => 100
Value of a => (a /= b) => 10
Value of a => (a %= b) => 0

Set the variables to different values and different operators and then
try...
```


tutorialspoint
SIMPLYEASYLEARNING

Miscellaneous Operators

We will discuss two operators here that are quite useful in JavaScript: the **conditional operator (? :)** and the **typeof operator**.

Conditional Operator (? :)

The conditional operator first evaluates an expression for a true or false value and then executes one of the two given statements depending upon the result of the evaluation.

S.No	Operator and Description
1	**? : (Conditional)** If Condition is true? Then value X : Otherwise value Y

Example

Try the following code to understand how the Conditional Operator works in JavaScript.

```
<html>
<body>

<script type="text/javascript">
<!-- var a
= 10; var
b = 20;
var linebreak = "<br />";

document.write ("((a > b) ? 100 : 200) => ");
result = (a > b) ? 100 : 200;
document.write(result);
document.write(linebreak);

document.write ("((a < b) ? 100 : 200) => ");
```


tutorialspoint
SIMPLYEASYLEARNING

```
result = (a < b) ? 100 : 200;
document.write(result);
document.write(linebreak);


//-->
</script>


<p>Set the variables to different values and different operators and
then try...</p>
</body>
</html>
```

Output

```
((a > b) ? 100 : 200) => 200
((a < b) ? 100 : 200) => 100
Set the variables to different values and different operators and then
try...
```

typeof Operator

The **typeof** operator is a unary operator that is placed before its single operand, which can be of any type. Its value is a string indicating the data type of the operand.

The *typeof* operator evaluates to "number", "string", or "boolean" if its operand is a number, string, or boolean value and returns true or false based on the evaluation.

Here is a list of the return values for the **typeof** Operator.

Type	String Returned by typeof
Number	"number"
String	"string"
Boolean	"boolean"
Object	"object"

tutorialspoint
SIMPLYEASYLEARNING

Function	"function"
Undefined	"undefined"
Null	"object"

Example

The following code shows how to implement **typeof** operator.

```
<html>
<body>

<script type="text/javascript">
<!-- var a = 10; var b =
"String"; var linebreak
= "<br />";

result = (typeof b == "string" ? "B is String" : "B is
Numeric"); document.write("Result => "); document.write(result);
document.write(linebreak);

result = (typeof a == "string" ? "A is String" : "A is
Numeric"); document.write("Result => "); document.write(result);
document.write(linebreak);

//-->
</script>

<p>Set the variables to different values and different operators and
then try...</p>
</body> </html>
```


tutorialspoint
SIMPLYEASYLEARNING

Output

```
Result => B is String
Result => A is Numeric

Set the variables to different values and different operators and then
try...
```


tutorialspoint
SIMPLYEASYLEARNING

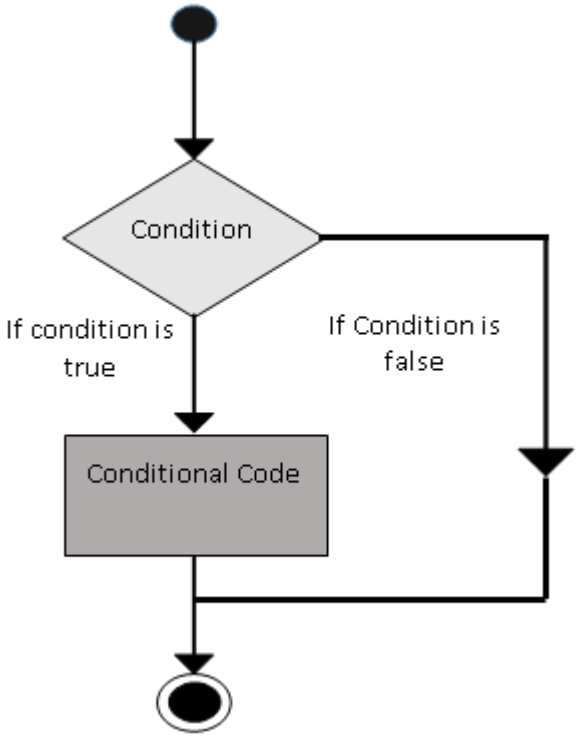
7. IF-ELSE

While writing a program, there may be a situation when you need to adopt one out of a given set of paths. In such cases, you need to use conditional statements that allow your program to make correct decisions and perform right actions.

JavaScript supports conditional statements which are used to perform different actions based on different conditions. Here we will explain the **if..else** statement.

Flow Chart of if-else

The following flow chart shows how the if-else statement works.



JavaScript supports the following forms of **if..else** statement:

- if statement
- if...else statement
- if...else if... statement

if Statement

tutorialspoint
SIMPLYEASYLEARNING

The '**if**' statement is the fundamental control statement that allows JavaScript to make decisions and execute statements conditionally.

Syntax

The syntax for a basic if statement is as follows:

```
if (expression){
   Statement(s) to be executed if expression is true }
```

Here a JavaScript expression is evaluated. If the resulting value is true, the given statement(s) are executed. If the expression is false, then no statement would be not executed. Most of the times, you will use comparison operators while making decisions.

Example

Try the following example to understand how the **if** statement works.

```
<html>
<body>

<script type="text/javascript">
<!-- var age =
20; if( age >
18 ){
   document.write("<b>Qualifies for driving</b>");
}
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Qualifies for driving
Set the variable to different value and then try...
```


tutorialspoint
SIMPLYEASYLEARNING

if...else Statement

The '**if...else**' statement is the next form of control statement that allows JavaScript to execute statements in a more controlled way.

Syntax

The syntax of an **if-else** statement is as follows:

```
if (expression){
   Statement(s) to be executed if expression is true
}else{
   Statement(s) to be executed if expression is false }
```

Here JavaScript expression is evaluated. If the resulting value is true, the given statement(s) in the 'if' block, are executed. If the expression is false, then the given statement(s) in the else block are executed.

Example

Try the following code to learn how to implement an if-else statement in JavaScript.

```
<html>
<body>

<script type="text/javascript">
<!-- var age = 15;

if( age > 18 ){
   document.write("<b>Qualifies for driving</b>");
}else{
   document.write("<b>Does not qualify for driving</b>"); }
```


tutorialspoint
SIMPLYEASYLEARNING

```
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Does not qualify for driving
Set the variable to different value and then try...
```

if...else if... Statement

The '**if...else if...**' statement is an advanced form of **if...else** that allows JavaScript to make a correct decision out of several conditions.

Syntax

The syntax of an if-else-if statement is as follows:

```
if (expression 1){
   Statement(s) to be executed if expression 1 is true }else if
(expression 2){
   Statement(s) to be executed if expression 2 is true }else if
(expression 3){
   Statement(s) to be executed if expression 3 is true
}else{
   Statement(s) to be executed if no expression is true }
```

There is nothing special about this code. It is just a series of **if** statements, where each **if** is a part of the **else** clause of the previous statement. Statement(s) are executed based on the true condition, if none of the conditions is true, then the **else** block is executed.

Example

tutorialspoint
SIMPLYEASYLEARNING

Try the following code to learn how to implement an if-else-if statement in JavaScript.

```
<html>
<body>

<script type="text/javascript">
<!--
var book = "maths"; if(
book == "history" ){
   document.write("<b>History Book</b>");
}else if( book == "maths" ){
   document.write("<b>Maths Book</b>"); }else
if( book == "economics" ){
   document.write("<b>Economics Book</b>");
}else{
  document.write("<b>Unknown Book</b>");
}
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Maths Book
Set the variable to different value and then try...
```


tutorialspoint
SIMPLYEASYLEARNING

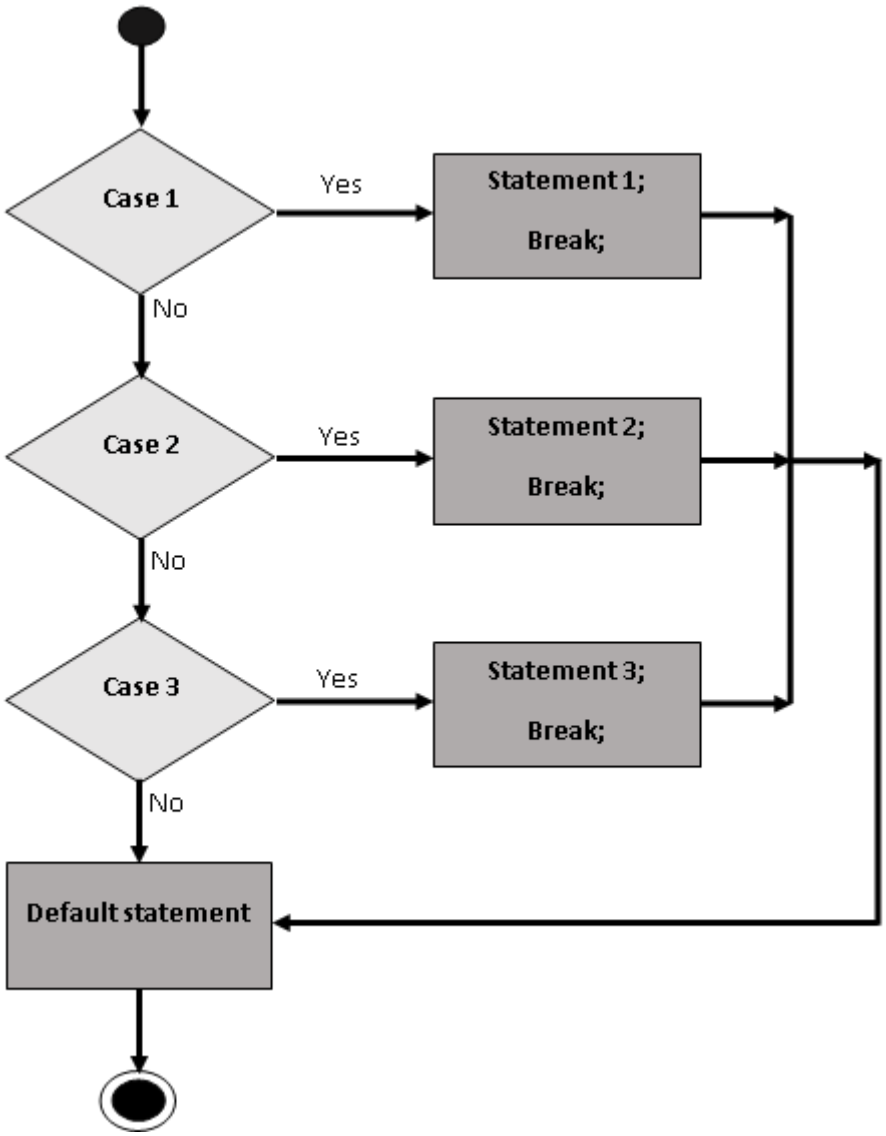
8. SWITCH-CASE

You can use multiple **if...else...if** statements, as in the previous chapter, to perform a multiway branch. However, this is not always the best solution, especially when all of the branches depend on the value of a single variable.

Starting with JavaScript 1.2, you can use a **switch** statement which handles exactly this situation, and it does so more efficiently than repeated **if...else if** statements.

Flow Chart

The following flow chart explains a switch-case statement works.



Syntax

The objective of a **switch** statement is to give an expression to evaluate and several different statements to execute based on the value of the expression. The interpreter checks each **case** against the value of the expression until a match is found. If nothing matches, a **default** condition will be used.

```
switch (expression)
{
  case condition 1: statement(s)                    break;   case
condition 2: statement(s)                    break;    ...   case
condition n: statement(s)                    break;   default:
statement(s)
}
```

The **break** statements indicate the end of a particular case. If they were omitted, the interpreter would continue executing each statement in each of the following cases.

We will explain **break** statement in **Loop Control** chapter.

Example

Try the following example to implement switch-case statement.

```
<html>
<body>

<script type="text/javascript">
<!-- var
grade='A';
document.write("Entering switch block<br />"); switch
(grade)
{
  case 'A': document.write("Good job<br />");
```


tutorialspoint
SIMPLYEASYLEARNING

```
            break;
  case 'B': document.write("Pretty good<br />");
break;
  case 'C': document.write("Passed<br />");
break;
  case 'D': document.write("Not so good<br />");
break;
  case 'F': document.write("Failed<br />");
break;
  default:  document.write("Unknown grade<br />")
}
document.write("Exiting switch block");
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Entering switch block
Good job
Exiting switch block
Set the variable to different value and then try...
```

Break statements play a major role in switch-case statements. Try the following code that uses switch-case statement without any break statement.

```
<html>
<body>

<script type="text/javascript">
<!-- var
grade='A';
document.write("Entering switch block<br />");
```


tutorialspoint
SIMPLYEASYLEARNING

```
switch (grade)
{
  case 'A': document.write("Good job<br />");
case 'B': document.write("Pretty good<br />");
case 'C': document.write("Passed<br />");   case
'D': document.write("Not so good<br />");   case
'F': document.write("Failed<br />");   default:
document.write("Unknown grade<br />")
}
document.write("Exiting switch block");
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Entering switch block
Good job
Pretty good
Passed
Not so good
Failed
Unknown grade
Exiting switch block
Set the variable to different value and then try...
```

While writing a program, you may encounter a situation where you need to perform an action over and over again. In such situations, you would need to write loop statements to reduce the number of lines.

JavaScript supports all the necessary loops to ease down the pressure of programming.

tutorialspoint
SIMPLYEASYLEARNING

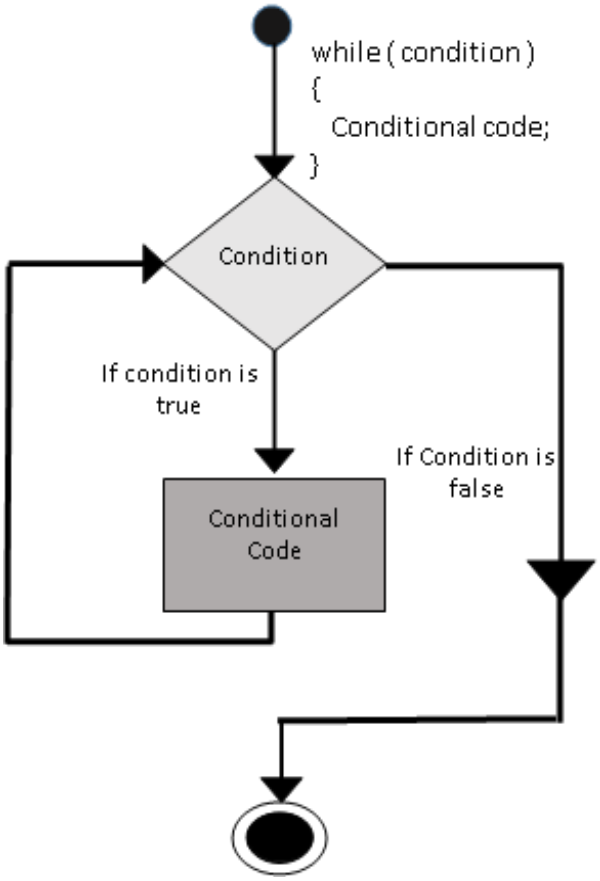
9. WHILE LOOP

The while Loop

The most basic loop in JavaScript is the **while** loop which would be discussed in this chapter. The purpose of a **while** loop is to execute a statement or code block repeatedly as long as an **expression** is true. Once the expression becomes **false**, the loop terminates.

Flow Chart

The flow chart of **while loop** looks as follows:



Syntax

The syntax of **while loop** in JavaScript is as follows:

```
while (expression){
   Statement(s) to be executed if expression is true
}
```

Example

Try the following example to implement while loop.

```
<html>
<body>

<script type="text/javascript">
<!-- var
count = 0;
document.write("Starting Loop "); while
(count < 10){
  document.write("Current Count : " + count + "<br />");
count++;
}
document.write("Loop stopped!");
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Starting Loop Current Count : 0
Current Count : 1
Current Count : 2
Current Count : 3
Current Count : 4
```


tutorialspoint
SIMPLYEASYLEARNING

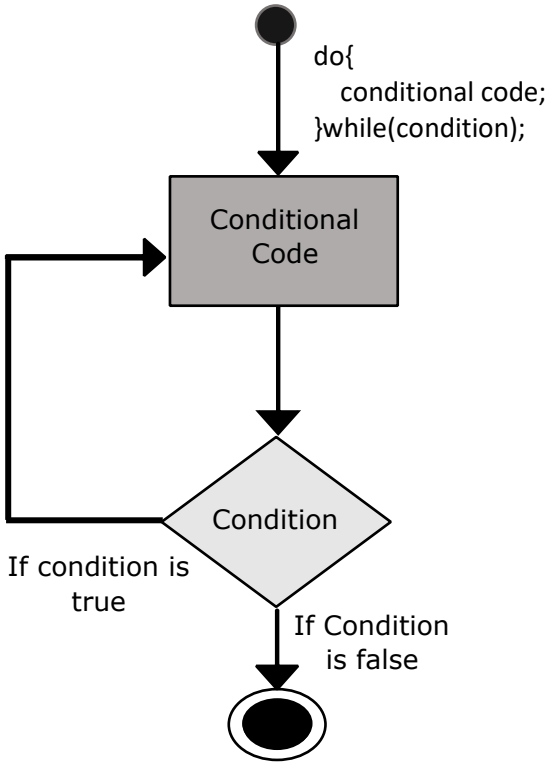
```
Current Count : 5
Current Count : 6
Current Count : 7
Current Count : 8
Current Count : 9 Loop
stopped!
Set the variable to different value and then try...
```

The do...while Loop

The **do...while** loop is similar to the **while** loop except that the condition check happens at the end of the loop. This means that the loop will always be executed at least once, even if the condition is **false**.

Flow Chart

The flow chart of a **do-while** loop would be as follows:



Syntax

The syntax for **do-while** loop in JavaScript is as follows:

```
do{
```


tutorialspoint
SIMPLYEASYLEARNING

```
   Statement(s) to be executed;
} while (expression);
```

Note: Don't miss the semicolon used at the end of the **do...while** loop.

Example

Try the following example to learn how to implement a **do-while** loop in JavaScript.

```
<html>
<body>

<script type="text/javascript">
<!-- var
count = 0;
document.write("Starting Loop" + "<br />"); do{
  document.write("Current Count : " + count + "<br />");
count++;
}while (count < 5);
document.write ("Loop stopped!");
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body>
</html>
```

Output

```
Starting Loop
Current Count : 0
Current Count : 1
Current Count : 2
Current Count : 3
Current Count : 4
Loop Stopped!
```

```
Set the variable to different value and then try...
```

10. FOR LOOP

The for Loop

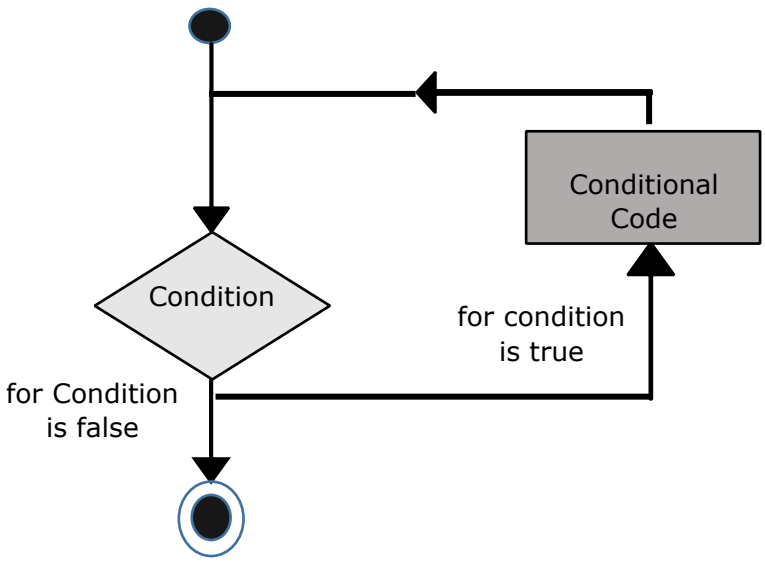
The '**for**' loop is the most compact form of looping. It includes the following three important parts:

- The **loop initialization** where we initialize our counter to a starting value. The initialization statement is executed before the loop begins.
- The **test statement** which will test if a given condition is true or not. If the condition is true, then the code given inside the loop will be executed, otherwise the control will come out of the loop.
- The **iteration statement** where you can increase or decrease your counter.

You can put all the three parts in a single line separated by semicolons.

Flow Chart

The flow chart of a **for** loop in JavaScript would be as follows:



tutorialspoint
SIMPLYEASYLEARNING

Syntax

The syntax of **for** loop is JavaScript is as follows:

```
for (initialization; test condition; iteration statement){
Statement(s) to be executed if test condition is true }
```

Example

Try the following example to learn how a **for** loop works in JavaScript.

```
<html>
<body>

<script type="text/javascript">
<!-- var
count;
document.write("Starting Loop" + "<br />"); for(count
= 0; count < 10; count++){   document.write("Current
Count : " + count );   document.write("<br />");
}
document.write("Loop stopped!");
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Starting Loop
Current Count : 0
Current Count : 1
Current Count : 2
Current Count : 3
Current Count : 4
Current Count : 5
```


tutorialspoint
SIMPLYEASYLEARNING

```
Current Count : 6
Current Count : 7
Current Count : 8
Current Count : 9 Loop
stopped!

Set the variable to different value and then try...
```


tutorialspoint
SIMPLYEASYLEARNING

11. FOR-IN LOOP

The **for...in** loop is used to loop through an object's properties. As we have not discussed Objects yet, you may not feel comfortable with this loop. But once you understand how objects behave in JavaScript, you will find this loop very useful.

Syntax

The syntax of 'for..in' loop is:

```
for (variablename in object){   statement or block to
execute
}
```

In each iteration, one property from **object** is assigned to **variablename** and this loop continues till all the properties of the object are exhausted.

Example

Try the following example to implement 'for-in' loop. It prints the web browser's **Navigator** object.

```
<html>
<body>

<script type="text/javascript">
<!-- var
aProperty;
document.write("Navigator Object Properties<br /> "); for
(aProperty in navigator)
{
  document.write(aProperty);
document.write("<br />");
}
document.write ("Exiting from the loop!");
//-->
</script>
```


tutorialspoint
SIMPLYEASYLEARNING

```
<p>Set the variable to different object and then try...</p>
</body> </html>
```

Output

```
Navigator Object
Properties serviceWorker
webkitPersistentStorage
webkitTemporaryStorage
geolocation doNotTrack
onLine languages language
userAgent product platform
appVersion appName
appCodeName
hardwareConcurrency
maxTouchPoints vendorSub
vendor productSub
cookieEnabled mimeTypes
plugins javaEnabled
getStorageUpdates
getGamepads
webkitGetUserMedia vibrate
getBattery sendBeacon
registerProtocolHandler
unregisterProtocolHandler Exiting
from the loop!
Set the variable to different object and then try...
```

JavaScript provides full control to handle loops and switch statements. There may be a situation when you need to come out of a loop without reaching at its bottom. There may also be a situation when you want to skip a part of your code block and start the next iteration of the look.

tutorialspoint
SIMPLYEASYLEARNING

12. LOOP CONTROL

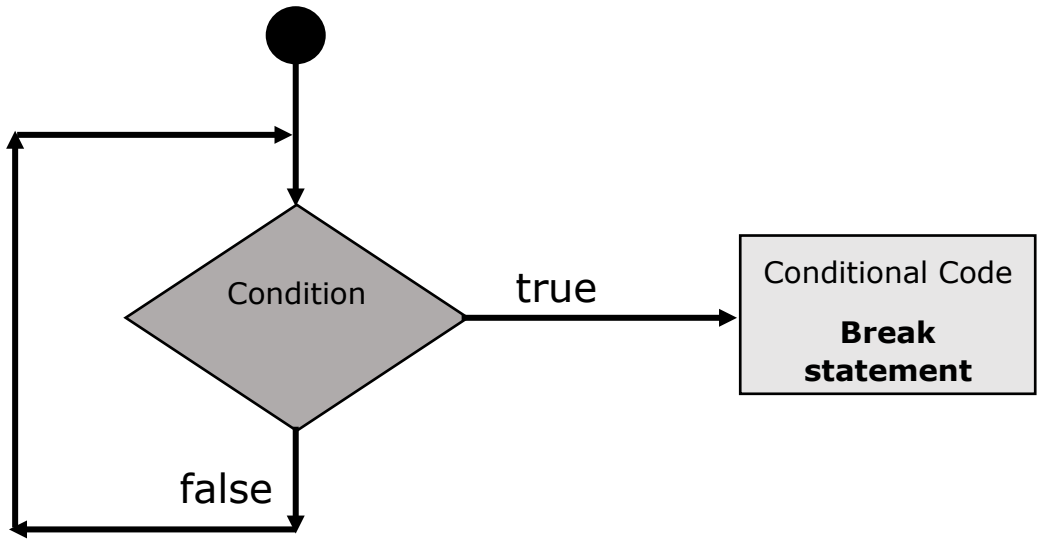
To handle all such situations, JavaScript provides **break** and **continue** statements. These statements are used to immediately come out of any loop or to start the next iteration of any loop respectively.

The break Statement

The **break** statement, which was briefly introduced with the *switch* statement, is used to exit a loop early, breaking out of the enclosing curly braces.

Flow Chart

The flow chart of a break statement would look as follows:



Example

The following example illustrates the use of a **break** statement with a while loop. Notice how the loop breaks out early once **x** reaches 5 and reaches to **document.write (..)** statement just below to the closing curly brace:

tutorialspoint
SIMPLYEASYLEARNING

```
<html>
<body>

<script type="text/javascript">
<!-- var
x = 1;
document.write("Entering the loop<br /> "); while
(x < 20)
{
  if (x == 5){
     break;  // breaks out of loop completely
  }   x = x
+ 1;
  document.write( x + "<br />");
}
document.write("Exiting the loop!<br /> ");
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

```
Entering the loop
2
3
4
5
Exiting the loop!
Set the variable to different value and then try...
```

We have already seen the usage of **break** statement inside a **switch** statement.

tutorialspoint
SIMPLYEASYLEARNING

The continue Statement

The **continue** statement tells the interpreter to immediately start the next iteration of the loop and skip the remaining code block. When a **continue** statement is encountered, the program flow moves to the loop check expression immediately and if the condition remains true, then it starts the next iteration, otherwise the control comes out of the loop.

Example

This example illustrates the use of a **continue** statement with a while loop. Notice how the **continue** statement is used to skip printing when the index held in variable **x** reaches 5.

```
<html>
<body>
<script type="text/javascript">
<!-- var
x = 1;
document.write("Entering the loop<br /> "); while
(x < 10)
{
  x = x + 1;
if (x == 5){
     continue;  // skill rest of the loop body
  }
  document.write( x + "<br />");
}
document.write("Exiting the loop!<br /> ");
//-->
</script>

<p>Set the variable to different value and then try...</p>
</body> </html>
```

Output

tutorialspoint
SIMPLYEASYLEARNING

```
Entering the loop
2 3
4
6 7
8 9
10
Exiting the loop!
```

Using Labels to Control the Flow

Starting from JavaScript 1.2, a label can be used with **break** and **continue** to control the flow more precisely. A **label** is simply an identifier followed by a colon (:) that is applied to a statement or a block of code. We will see two different examples to understand how to use labels with break and continue.

Note: Line breaks are not allowed between the '**continue**' or '**break**' statement and its label name. Also, there should not be any other statement in between a label name and associated loop.

Try the following two examples for a better understanding of Labels.

Example 1

The following example shows how to implement Label with a break statement.

```
<html>
<body>
<script type="text/javascript">
<!--
document.write("Entering the loop!<br />
"); outerloop:   // This is the label name
for (var i = 0; i < 5; i++)
{
  document.write("Outerloop: " + i + "<br />");
innerloop:
  for (var j = 0; j < 5; j++)
```


tutorialspoint
SIMPLYEASYLEARNING

```
  {
     if (j >  3 ) break ;          // Quit the innermost
loop      if (i == 2) break innerloop; // Do the same thing
if (i == 4) break outerloop; // Quit the outer loop
document.write("Innerloop: " + j + "  <br />");
   } }
document.write("Exiting the loop!<br /> ");
//-->
</script>
</body> </html>
```

Output

```
Entering the loop!
Outerloop: 0
Innerloop: 0
Innerloop: 1
Innerloop: 2
Innerloop: 3  Outerloop: 1
Innerloop: 0
Innerloop: 1
Innerloop: 2
Innerloop: 3
Outerloop: 2
Outerloop: 3
Innerloop: 0
Innerloop: 1
Innerloop: 2
Innerloop: 3
Outerloop: 4 Exiting the loop!
```

Example 2

The following example shows how to implement Label with continue.

```
<html>
```


tutorialspoint
SIMPLYEASYLEARNING

```
<body>
<script type="text/javascript">
<!--
document.write("Entering the loop!<br />
"); outerloop:   // This is the label name
for (var i = 0; i < 3; i++)
{
  document.write("Outerloop: " + i + "<br />");
for (var j = 0; j < 5; j++)
  {
    if (j == 3){
continue outerloop;
     }
    document.write("Innerloop: " + j + "<br />");
   }  }
document.write("Exiting the loop!<br /> ");
//-->
</script>

</body> </html>
```

Output

```
Entering the loop! Outerloop: 0
Innerloop: 0
Innerloop: 1
Innerloop: 2
Outerloop: 1
Innerloop: 0
Innerloop: 1
Innerloop: 2
Outerloop: 2
Innerloop: 0
Innerloop: 1 Innerloop: 2 Exiting the loop!
```


tutorialspoint
SIMPLYEASYLEARNING

tutorialspoint
SIMPLYEASYLEARNING

13. FUNCTIONS

A function is a group of reusable code which can be called anywhere in your program. This eliminates the need of writing the same code again and again. It helps programmers in writing modular codes. Functions allow a programmer to divide a big program into a number of small and manageable functions.

Like any other advanced programming language, JavaScript also supports all the features necessary to write modular code using functions. You must have seen functions like **alert()** and **write()** in the earlier chapters. We were using these functions again and again, but they had been written in core JavaScript only once.

JavaScript allows us to write our own functions as well. This section explains how to write your own functions in JavaScript.

Function Definition

Before we use a function, we need to define it. The most common way to define a function in JavaScript is by using the **function** keyword, followed by a unique function name, a list of parameters (that might be empty), and a statement block surrounded by curly braces.

Syntax

The basic syntax is shown here.

```
<script type="text/javascript">
<!--
function functionname(parameter-list)
{   statements
}
//--> </script>
```

Example

Try the following example. It defines a function called sayHello that takes no parameters:

```
<script type="text/javascript">
<!--
function sayHello()
{
```

```
   alert("Hello there");
}
//-->
</script>
```

Calling a Function

To invoke a function somewhere later in the script, you would simply need to write the name of that function as shown in the following code.

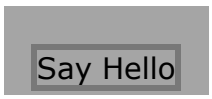
```
<html>
<head>
<script type="text/javascript">  function
sayHello()
{
  document.write ("Hello there!");
}
</script>
</head>

<body>
<p>Click the following button to call the function</p>
<form>
<input type="button" onclick="sayHello()" value="Say Hello">
</form>

<p>Use different text in write method and then try...</p>
</body> </html>
```

Output

Click the following button to call the function



tutorialspoint
SIMPLYEASYLEARNING

Function Parameters

Till now, we have seen functions without parameters. But there is a facility to pass different parameters while calling a function. These passed parameters can be captured inside the function and any manipulation can be done over those parameters. A function can take multiple parameters separated by comma.

Example

Try the following example. We have modified our **sayHello** function here. Now it takes two parameters.

```
<html>
<head>
<script type="text/javascript">  function
sayHello(name, age)
{
   document.write (name + " is " + age + " years old."); }
</script>
</head>

<body>
<p>Click the following button to call the function</p>
<form>
<input type="button" onclick="sayHello('Zara', 7)" value="Say Hello">
</form>

<p>Use different parameters inside the function and then try...</p>
</body> </html>
```

Output

tutorialspoint
SIMPLYEASYLEARNING

```
Click the following button to call the function
```

Say Hello

```
Use different parameters inside the function and then try...
```

The return Statement

A JavaScript function can have an optional **return** statement. This is required if you want to return a value from a function. This statement should be the last statement in a function.

For example, you can pass two numbers in a function and then you can expect the function to return their multiplication in your calling program.

Example

Try the following example. It defines a function that takes two parameters and concatenates them before returning the resultant in the calling program.

```
<html>
<head>
<script type="text/javascript">
function concatenate(first, last)
{
   var full;

   full   =   first   +   last;
return  full;
}
function secondFunction()
{
   var result;
   result = concatenate('Zara', 'Ali');
document.write (result );
}
</script>
</head>
```


tutorialspoint
SIMPLYEASYLEARNING

```
<body>
<p>Click the following button to call the function</p>
<form>
<input type="button" onclick="secondFunction()" value="Call Function">
</form>

<p>Use different parameters inside the function and then try...</p>
</body> </html>
```

Output

```
Click the following button to call the function
```

Call Function

```
Use different parameters inside the function and then try...
```

There is a lot to learn about JavaScript functions, however we have covered the most important concepts in this tutorial.

Nested Functions

Prior to JavaScript 1.2, function definition was allowed only in top level global code, but JavaScript 1.2 allows function definitions to be nested within other functions as well. Still there is a restriction that function definitions may not appear within loops or conditionals. These restrictions on function definitions apply only to function declarations with the function statement.

As we'll discuss later in the next chapter, function literals (another feature introduced in JavaScript 1.2) may appear within any JavaScript expression, which means that they can appear within **if** and other statements.

Example

Try the following example to learn how to implement nested functions.

```
<html>
```



```
<head>
<script type="text/javascript">
<!--
function hypotenuse(a, b) {
function square(x) { return x*x; }

   return Math.sqrt(square(a) + square(b));
}
function secondFunction(){
var result;
   result = hypotenuse(1,2);
document.write ( result );
}
//-->
</script>
</head>

<body>
<p>Click the following button to call the function</p>

<form>
<input type="button" onclick="secondFunction()" value="Call Function">
</form>

<p>Use different parameters inside the function and then try...</p>
</body> </html>
```

Output

tutorialspoint
SIMPLYEASYLEARNING

```
Click the following button to call the function
```

Call Function

```
Use different parameters inside the function and then try...
```

Function () Constructor

The *function* statement is not the only way to define a new function; you can define your function dynamically using **Function()** constructor along with the **new** operator.

Note: Constructor is a terminology from Object Oriented Programming. You may not feel comfortable for the first time, which is OK.

Syntax

Following is the syntax to create a function using **Function()** constructor along with the **new** operator.

```
<script type="text/javascript">
<!--
var variablename = new Function(Arg1, Arg2..., "Function Body");
//-->
</script>
```

The **Function()** constructor expects any number of string arguments. The last argument is the body of the function – it can contain arbitrary JavaScript statements, separated from each other by semicolons.

Notice that the **Function()** constructor is not passed any argument that specifies a name for the function it creates. The **unnamed** functions created with the **Function()** constructor are called **anonymous** functions.

Example

Try the following example.

```
<html>
<head>
<script type="text/javascript">
<!--
var func = new Function("x", "y", "return x*y;");

function secondFunction(){
var result;    result =
func(10,20);
```



```
   document.write ( result );
}
//-->
</script>
</head>

<body>
<p>Click the following button to call the function</p>

<form>
<input type="button" onclick="secondFunction()" value="Call Function">
</form>

<p>Use different parameters inside the function and then try...</p>
</body> </html>
```

Output



Click the following button to call the function

Use different parameters inside the function and then try...

Function Literals

tutorialspoint
SIMPLYEASYLEARNING

JavaScript 1.2 introduces the concept of **function literals** which is another new way of defining functions. A function literal is an expression that defines an unnamed function.

Syntax

The syntax for a **function literal** is much like a function statement, except that it is used as an expression rather than a statement and no function name is required.

```
<script type="text/javascript">
<!--
```

```
var variablename = function(Argument List){
                        Function Body
   }; //-->
</script>
```

Syntactically, you can specify a function name while creating a literal function as follows.

```
<script type="text/javascript">
<!--
var variablename = function FunctionName(Argument List){
                        Function Body
       }; //-->
</script>
```

But this name does not have any significance, so it is not worthwhile. **Example**

Try the following example. It shows the usage of function literals.

```
<html>
<head>
<script type="text/javascript">
<!--
var func = function(x,y){ return x*y };

function secondFunction(){
var result;    result =
func(10,20);
document.write ( result );
}
//-->
</script>
</head>
<body>
```

72

```
<p>Click the following button to call the function</p>
<form>
<input type="button" onclick="secondFunction()" value="Call Function">
</form>
<p>Use different parameters inside the function and then try...</p>
</body> </html>
```

Output

```
Click the following button to call the function
```




```
Use different parameters inside the function and then try...
```


tutorialspoint
SIMPLYEASYLEARNING

14. EVENTS

What is an Event?

JavaScript's interaction with HTML is handled through events that occur when the user or the browser manipulates a page.

When the page loads, it is called an event. When the user clicks a button, that click too is an event. Other examples include events like pressing any key, closing a window, resizing a window, etc.

Developers can use these events to execute JavaScript coded responses, which cause buttons to close windows, messages to be displayed to users, data to be validated, and virtually any other type of response imaginable.

Events are a part of the Document Object Model (DOM) Level 3 and every HTML element contains a set of events which can trigger JavaScript Code.

Please go through this small tutorial for a better understanding HTML Event Reference. Here we will see a few examples to understand the relation between Event and JavaScript.

onclick Event Type

This is the most frequently used event type which occurs when a user clicks the left button of his mouse. You can put your validation, warning etc., against this event type.

Example

Try the following example.

```
<html>
<head>
<script type="text/javascript">
<!--
function sayHello() {    document.write ("Hello World")
}
//-->
</script>
```


tutorialspoint
SIMPLYEASYLEARNING

```
</head>
<body>
<p> Click the following button and see result</p>
<input type="button" onclick="sayHello()" value="Say Hello" />
</body> </html>
```

Output

```
Click the following button and see result
```

Say Hello

onsubmit Event Type

---

onsubmit is an event that occurs when you try to submit a form. You can put your form validation against this event type.

Example

The following example shows how to use onsubmit. Here we are calling a **validate()** function before submitting a form data to the webserver. If **validate()** function returns true, the form will be submitted, otherwise it will not submit the data.

Try the following example.

```
<html>
<head>
<script type="text/javascript">
<!--
function validation() {    all validation goes here    .........
return either true or false
}
//-->
</script>
</head>
```


tutorialspoint
SIMPLYEASYLEARNING

```
<body>
<form method="POST" action="t.cgi" onsubmit="return validate()"> .......
<input type="submit" value="Submit" />
</form> </body>
</html>
```

onmouseover and onmouseout

These two event types will help you create nice effects with images or even with text as well. The **onmouseover** event triggers when you bring your mouse over any element and the **onmouseout** triggers when you move your mouse out from that element. Try the following example.

```
<html>
<head>
<script type="text/javascript">
<!--
function over() {
   document.write ("Mouse Over");
}
function out() {
   document.write ("Mouse Out");
}
//-->
</script>
</head>
<body>
<p>Bring your mouse inside the division to see the result:</p>
<div onmouseover="over()" onmouseout="out()">
<h2> This is inside the division </h2>
</div>
</body> </html>
```


tutorialspoint
SIMPLYEASYLEARNING

Output

```
Bring your mouse inside the division to see the result:

This is inside the division
```

HTML 5 Standard Events

The standard HTML 5 events are listed here for your reference.
Here script indicates a Javascript function to be executed against that event.

Attribute	Value	Description
Offline	script	Triggers when the document goes offline
Onabort	script	Triggers on an abort event
onafterprint	script	Triggers after the document is printed
onbeforeonload	script	Triggers before the document loads
onbeforeprint	script	Triggers before the document is printed
onblur	script	Triggers when the window loses focus
oncanplay	script	Triggers when media can start play, but might has to stop for buffering
oncanplaythrough	script	Triggers when media can be played to the end, without stopping for buffering
onchange	script	Triggers when an element changes
onclick	script	Triggers on a mouse click
oncontextmenu	script	Triggers when a context menu is triggered

tutorialspoint
SIMPLYEASYLEARNING

ondblclick	script	Triggers on a mouse double-click
ondrag	script	Triggers when an element is dragged
ondragend	script	Triggers at the end of a drag operation

ondragenter	script	Triggers when an element has been dragged to a valid drop target
ondragleave	script	Triggers when an element leaves a valid drop target
ondragover	script	Triggers when an element is being dragged over a valid drop target
ondragstart	script	Triggers at the start of a drag operation
ondrop	script	Triggers when dragged element is being dropped
ondurationchange	script	Triggers when the length of the media is changed
onemptied	script	Triggers when a media resource element suddenly becomes empty.
onended	script	Triggers when media has reach the end
onerror	script	Triggers when an error occur
onfocus	script	Triggers when the window gets focus
onformchange	script	Triggers when a form changes

tutorialspoint
SIMPLYEASYLEARNING

onforminput	script	Triggers when a form gets user input
onhaschange	script	Triggers when the document has change
oninput	script	Triggers when an element gets user input
oninvalid	script	Triggers when an element is invalid
onkeydown	script	Triggers when a key is pressed
onkeypress	script	Triggers when a key is pressed and released
onkeyup	script	Triggers when a key is released

onload	script	Triggers when the document loads
onloadeddata	script	Triggers when media data is loaded
onloadedmetadata	script	Triggers when the duration and other media data of a media element is loaded
onloadstart	script	Triggers when the browser starts to load the media data
onmessage	script	Triggers when the message is triggered
onmousedown	script	Triggers when a mouse button is pressed
onmousemove	script	Triggers when the mouse pointer moves
onmouseout	script	Triggers when the mouse pointer moves out of an element

tutorialspoint
SIMPLYEASYLEARNING

onmouseover	script	Triggers when the mouse pointer moves over an element
onmouseup	script	Triggers when a mouse button is released
onmousewheel	script	Triggers when the mouse wheel is being rotated
onoffline	script	Triggers when the document goes offline
onoine	script	Triggers when the document comes online
ononline	script	Triggers when the document comes online
onpagehide	script	Triggers when the window is hidden
onpageshow	script	Triggers when the window becomes visible
onpause	script	Triggers when media data is paused
onplay	script	Triggers when media data is going to start playing

onplaying	script	Triggers when media data has start playing
onpopstate	script	Triggers when the window's history changes
onprogress	script	Triggers when the browser is fetching the media data
onratechange	script	Triggers when the media data's playing rate has changed
onreadystatechange	script	Triggers when the ready-state changes

tutorialspoint
SIMPLYEASYLEARNING

onredo	script	Triggers when the document performs a redo
onresize	script	Triggers when the window is resized
onscroll	script	Triggers when an element's scrollbar is being scrolled
onseeked	script	Triggers when a media element's seeking attribute is no longer true, and the seeking has ended
onseeking	script	Triggers when a media element's seeking attribute is true, and the seeking has begun
onselect	script	Triggers when an element is selected
onstalled	script	Triggers when there is an error in fetching media data
onstorage	script	Triggers when a document loads
onsubmit	script	Triggers when a form is submitted
onsuspend	script	Triggers when the browser has been fetching media data, but stopped before the entire media file was fetched
ontimeupdate	script	Triggers when media changes its playing position
onundo	script	Triggers when a document performs an undo
onunload	script	Triggers when the user leaves the document
onvolumechange	script	Triggers when media changes the volume, also when volume is set to "mute"

onwaiting	script	Triggers when media has stopped playing, but is expected to resume

tutorialspoint
SIMPLYEASYLEARNING

15. COOKIES

What are Cookies?

Web Browsers and Servers use HTTP protocol to communicate and HTTP is a stateless protocol. But for a commercial website, it is required to maintain session information among different pages. For example, one user registration ends after completing many pages. But how to maintain users' session information across all the web pages.

In many situations, using cookies is the most efficient method of remembering and tracking preferences, purchases, commissions, and other information required for better visitor experience or site statistics.

How It Works?

Your server sends some data to the visitor's browser in the form of a cookie. The browser may accept the cookie. If it does, it is stored as a plain text record on the visitor's hard drive. Now, when the visitor arrives at another page on your site, the browser sends the same cookie to the server for retrieval. Once retrieved, your server knows/remembers what was stored earlier.

Cookies are a plain text data record of 5 variable-length fields:

- **Expires:** The date the cookie will expire. If this is blank, the cookie will expire when the visitor quits the browser.
- **Domain:** The domain name of your site.
- **Path:** The path to the directory or web page that set the cookie. This may be blank if you want to retrieve the cookie from any directory or page.
- **Secure:** If this field contains the word "secure", then the cookie may only be retrieved with a secure server. If this field is blank, no such restriction exists.
- **Name=Value:** Cookies are set and retrieved in the form of key-value pairs.

Cookies were originally designed for CGI programming. The data contained in a cookie is automatically transmitted between the web browser and the web server, so CGI scripts on the server can read and write cookie values that are stored on the client.

JavaScript can also manipulate cookies using the **cookie** property of the **Document** object. JavaScript can read, create, modify, and delete the cookies that apply to the current web page.

Storing Cookies

The simplest way to create a cookie is to assign a string value to the document.cookie object, which looks like this.

```
document.cookie = "key1=value1;key2=value2;expires=date";
```

Here the **expires** attribute is optional. If you provide this attribute with a valid date or time, then the cookie will expire on a given date or time and thereafter, the cookies' value will not be accessible.

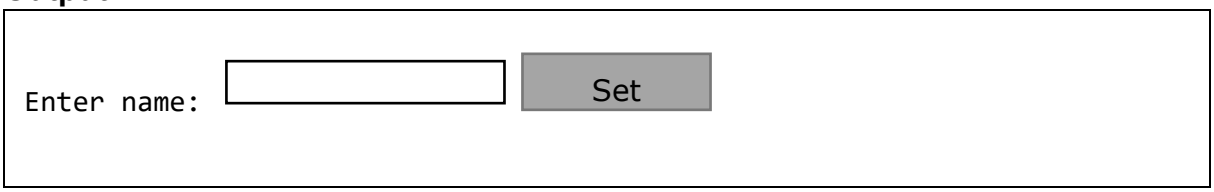
Note: Cookie values may not include semicolons, commas, or whitespace. For this reason, you may want to use the JavaScript **escape()** function to encode the value before storing it in the cookie. If you do this, you will also have to use the corresponding **unescape()** function when you read the cookie value. **Example**

Try the following. It sets a customer name in an input cookie.

```
<html>
<head>
<script type="text/javascript">
<!--
function WriteCookie()
{
   if( document.myform.customer.value == "" ){
alert ("Enter some value!");       return;
   }
   cookievalue= escape(document.myform.customer.value) + ";";
document.cookie="name=" + cookievalue;
   document.write ("Setting Cookies : " + "name=" + cookievalue ); }
//-->
</script>
```

```
</head>
<body>
<form name="myform" action="">
Enter name: <input type="text" name="customer"/>
<input type="button" value="Set Cookie" onclick="WriteCookie();"/>
</form> </body>
</html>
```

Output



Now your machine has a cookie called **name**. You can set multiple cookies using multiple key=value pairs separated by comma.

Reading Cookies

Reading a cookie is just as simple as writing one, because the value of the document.cookie object is the cookie. So you can use this string whenever you want to access the cookie. The document.cookie string will keep a list of name=value pairs separated by semicolons, where **name** is the name of a cookie and value is its string value.

You can use strings' **split()** function to break a string into key and values as follows:

Example

Try the following example to get all the cookies.

```
<html>
<head>
<script type="text/javascript">
<!--
function ReadCookie()
{
   var allcookies = document.cookie;
```

```
   document.write ("All Cookies : " + allcookies );
```



```
   //  Get  all  the  cookies  pairs  in  an  array
cookiearray  = allcookies.split(';');

   // Now take key value pair out of this array
for(var i=0; i<cookiearray.length; i++){
name = cookiearray[i].split('=')[0];       value
= cookiearray[i].split('=')[1];
      document.write ("Key is : " + name + " and Value is : " + value);
}
}
//-->
</script>
</head>
<body>
<form name="myform" action="">
<p> click the following button and see the result:</p>
<input type="button" value="Get Cookie" onclick="ReadCookie()"/>
</form> </body>
</html>
```

Note: Here **length** is a method of **Array** class which returns the length of an array. We will discuss Arrays in a separate chapter. By that time, please try to digest it.

Output

Get Cookie click the following button and see the result:

Note: There may be some other cookies already set on your machine. The above code will display all the cookies set on your machine.

Setting Cookies Expiry Date

You can extend the life of a cookie beyond the current browser session by setting an expiration date and saving the expiry date within the cookie. This can be done by setting the '**expires**' attribute to a date and time.

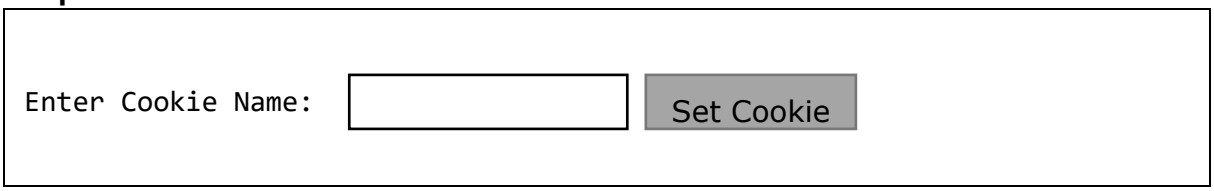
Example

Try the following example. It illustrates how to extend the expiry date of a cookie by 1 Month.

```
<html>
<head>
<script type="text/javascript">
<!--
function WriteCookie()
{
   var now = new Date();
   now.setMonth( now.getMonth() + 1 );
   cookievalue = escape(document.myform.customer.value) + ";"
document.cookie="name=" + cookievalue;
   document.cookie   =   "expires="   +   now.toUTCString()   +   ";"
document.write ("Setting Cookies : " + "name=" + cookievalue ); }
//-->
</script>
</head>
<body>
<form name="formname" action="">
Enter name: <input type="text" name="customer"/>
<input type="button" value="Set Cookie" onclick="WriteCookie()"/>
</form>
</body> </html>
```


tutorialspoint
SIMPLYEASYLEARNING

Output



Deleting a Cookie

Sometimes you will want to delete a cookie so that subsequent attempts to read the cookie return nothing. To do this, you just need to set the expiry date to a time in the past.

Example

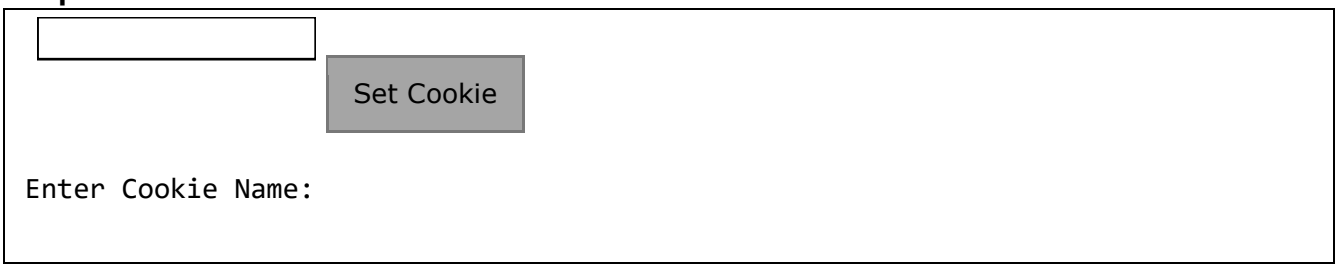
Try the following example. It illustrates how to delete a cookie by setting its expiry date to one month behind the current date.

```
<html>
<head>
<script type="text/javascript">
<!--
function WriteCookie()
{
   var now = new Date();
   now.setMonth( now.getMonth() - 1 );
   cookievalue = escape(document.myform.customer.value) + ";"
document.cookie="name=" + cookievalue;
   document.cookie = "expires=" + now.toUTCString() + ";"
document.write("Setting Cookies : " + "name=" + cookievalue );
}
//-->
</script>
</head>
<body>
<form name="formname" action="">
Enter name: <input type="text" name="customer"/>
<input type="button" value="Set Cookie" onclick="WriteCookie()"/>
```


tutorialspoint
SIMPLYEASYLEARNING

```
</form>
</body>
</html>
```

Output



tutorialspoint
SIMPLYEASYLEARNING

16. PAGE REDIRECT

What is Page Redirection?

You might have encountered a situation where you clicked a URL to reach a page X but internally you were directed to another page Y. It happens due to **page redirection**. This concept is different from JavaScript Page Refresh.

There could be various reasons why you would like to redirect a user from the original page. We are listing down a few of the reasons:

- You did not like the name of your domain and you are moving to a new one. In such a scenario, you may want to direct all your visitors to the new site. Here you can maintain your old domain but put a single page with a page redirection such that all your old domain visitors can come to your new domain.
- You have built-up various pages based on browser versions or their names or may be based on different countries, then instead of using your server-side page redirection, you can use client-side page redirection to land your users on the appropriate page.
- The Search Engines may have already indexed your pages. But while moving to another domain, you would not like to lose your visitors coming through search engines. So you can use client-side page redirection. But keep in mind this should not be done to fool the search engine, it could lead your site to get banned.

JavaScript Page Refresh

You can refresh a web page using JavaScript **location.reload** method. This code can be called automatically upon an event or simply when the user clicks on a link. If you want to refresh a web page using a mouse click, then you can use the following code:

```
<a href="javascript:location.reload(true)">Refresh Page</a>
```

Auto Refresh

tutorialspoint
SIMPLYEASYLEARNING

You can also use JavaScript to refresh the page automatically after a given time period. Here **setTimeout()** is a built-in JavaScript function which can be used to execute another function after a given time interval.

Example

Try the following example. It shows how to refresh a page after every 5 seconds. You can change this time as per your requirement.

```
<html>
<head>
<script type="text/JavaScript">
<!--
function AutoRefresh( t ) {
     setTimeout("location.reload(true);", t);
}
//   -->
</script>
</head>
<body onload="JavaScript:AutoRefresh(5000);">
<p>This page will refresh every 5 seconds.</p>
</body> </html>
```

Output

```
This page will refresh every 5 seconds.
```

How Page Re-direction Works?

The implementations of Page-Redirection are as follows.

Example 1

It is quite simple to do a page redirect using JavaScript at client side. To redirect your site visitors to a new page, you just need to add a line in your head section as follows.

```
<html>
<head>
```


tutorialspoint
SIMPLYEASYLEARNING

```
<script type="text/javascript">
<!--
function Redirect() {
    window.location="http://www.tutorialspoint.com";
}
//-->
</script>
</head>
<body>
<p>Click the following button, you will be redirected to home page.</p>
<form>
<input type="button" value="Redirect Me" onclick="Redirect();" />
</form>
</body> </html>
```

Output

Click the following button, you will be redirected to home page.

Redirect Me

Example 2

You can show an appropriate message to your site visitors before redirecting them to a new page. This would need a bit time delay to load a new page. The following example shows how to implement the same. Here **setTimeout()** is a built-in JavaScript function which can be used to execute another function after a given time interval.

```
<html>
<head>
<script type="text/javascript">
<!--
function Redirect() {
    window.location="http://www.tutorialspoint.com";
```

```
}
document.write ("You will be redirected to our main page in 10
seconds!");
```

```
setTimeout('Redirect()', 10000);
//-->
</script>
</head> <body>
</body> </html>
```

Output

```
You will be redirected to tutorialspoint.com main page in 10 seconds!
```

Example 3

The following example shows how to redirect your site visitors onto a different page based on their browsers.

tutorialspoint
SIMPLYEASYLEARNING

```
<html>
<head>
<script type="text/javascript">
<!--
var browsername=navigator.appName;  if(
browsername == "Netscape" )
{
   window.location="http://www.location.com/ns.htm";
}
else if ( browsername =="Microsoft Internet Explorer")
{
   window.location="http://www.location.com/ie.htm";
} else
{
  window.location="http://www.location.com/other.htm";
}
//-->
</script>
```

```
</head> <body>
</body>
</html>
```


tutorialspoint
SIMPLYEASYLEARNING

17. DIALOG BOX

JavaScript supports three important types of dialog boxes. These dialog boxes can be used to raise and alert, or to get confirmation on any input or to have a kind of input from the users. Here we will discuss each dialog box one by one.

Alert Dialog Box

An alert dialog box is mostly used to give a warning message to the users. For example, if one input field requires to enter some text but the user does not provide any input, then as a part of validation, you can use an alert box to give a warning message.

Nonetheless, an alert box can still be used for friendlier messages. Alert box gives only one button "OK" to select and proceed.

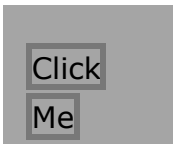
Example

```
<html>
<head>
<script type="text/javascript">
<!--
function Warn() {
   alert ("This is a warning message!");
document.write ("This is a warning message!");
}
//-->
</script>
</head>
<body>
<p>Click the following button to see the result: </p>
<form>
<input type="button" value="Click Me" onclick="Warn();" />
</form>
</body> </html>
```

Output

tutorialspoint
SIMPLYEASYLEARNING

```
Click the following button to see the result:
```



Confirmation Dialog Box

A confirmation dialog box is mostly used to take user's consent on any option. It displays a dialog box with two buttons: **OK** and **Cancel**.

If the user clicks on the OK button, the window method **confirm()** will return true. If the user clicks on the Cancel button, then **confirm()** returns false. You can use a confirmation dialog box as follows.

Example

```
<html>
<head>
<script type="text/javascript">
<!--
function getConfirmation(){
   var retVal = confirm("Do you want to continue ?");    if( retVal ==
true ){
      document.write ("User wants to continue!");      return true;
   }else{
      Document.write ("User does not want to continue!");          return
false;
   }
}
//-->
</script>
</head>
<body>
<p>Click the following button to see the result: </p>
<form>
```


tutorialspoint
SIMPLYEASYLEARNING

```
<input type="button" value="Click Me" onclick="getConfirmation();" />
</form> </body>
</html>
```

Output



Click the following button to see the result:

Prompt Dialog Box

The prompt dialog box is very useful when you want to pop-up a text box to get user input. Thus, it enables you to interact with the user. The user needs to fill in the field and then click OK.

This dialog box is displayed using a method called **prompt()** which takes two parameters: (i) a label which you want to display in the text box and (ii) a default string to display in the text box.

This dialog box has two buttons: **OK** and **Cancel**. If the user clicks the OK button, the window method **prompt()** will return the entered value from the text box. If the user clicks the Cancel button, the window method **prompt()** returns **null**.

Example

The following example shows how to use a prompt dialog box:

```
<html>
<head>
<script type="text/javascript">
<!--
function getValue(){
   var retVal = prompt("Enter your name : ", "your name here");

   document.write("You have entered : " +  retVal);
}
//-->
```


tutorialspoint
SIMPLYEASYLEARNING

```
</script>
</head>
<body>
<p>Click the following button to see the result: </p>
<form>
<input type="button" value="Click Me" onclick="getValue();" />
</form> </body>
</html>
```

Output



Click the following button to see the result:

18. VOID KEYWORD

void is an important keyword in JavaScript which can be used as a unary operator that appears before its single operand, which may be of any type. This operator specifies an expression to be evaluated without returning a value.

Syntax

The syntax of **void** can be either of the following two:

```
<head>
<script type="text/javascript">
<!-- void
func()
javascript:void
func()  OR
void(func())
javascript:void(func())
//-->
</script>
</head>
```

Example 1

The most common use of this operator is in a client-side *javascript: URL*, where it allows you to evaluate an expression for its side-effects without the browser displaying the value of the evaluated expression.

Here the expression **alert ('Warning!!!')** is evaluated but it is not loaded back into the current document:

```
<html>
<head>
<script type="text/javascript">
<!-- //-->
</script>
</head>

<body>
<p>Click the following, This won't react at all...</p>
<a href="javascript:void(document.write("Hello : 0"))">Click me!</a>
```


tutorialspoint
SIMPLYEASYLEARNING

```
</body>
</html>
```

Output

```
Click the following, This won't react at all...
Click me!
```

Example 2

Take a look at the following example. The following link does nothing because the expression "0" has no effect in JavaScript. Here the expression "0" is evaluated, but it is not loaded back into the current document.

```
<html>
<head>
<script type="text/javascript">
<!-- //-->
</script>
</head>
<body>
<p>Click the following, This won't react at all...</p>
<a href="javascript:void(0)">Click me!</a>
</body>
</html>
```

Output

```
Click the following, This won't react at all...
Click me!
```

Example 3

Another use of **void** is to purposely generate the **undefined** value as follows.

```
<html>
```

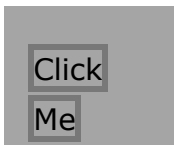

tutorialspoint
SIMPLYEASYLEARNING

```
<head>
<script type="text/javascript">
<!--
function getValue(){
var a,b,c;

   a = void ( b = 5, c = 7 );
   document.write('a = ' + a + ' b = ' + b +' c = ' + c ); }
//-->
</script>
</head>
<body>
<p>Click the following to see the result:</p>
<form>
<input type="button" value="Click Me" onclick="getValue();" />
</body> </html>
```

Output

Click the following button to see the result:



19. PAGE PRINTING

Many times you would like to place a button on your webpage to print the content of that web page via an actual printer. JavaScript helps you to implement this functionality using the **print** function of **window** object.

The JavaScript print function **window.print()** prints the current web page when executed. You can call this function directly using the **onclick** event as shown in the following example.

Example

Try the following example.

```
<head>
<script type="text/javascript">
<!-- //-->
</script>
</head>
<body>
<form>
<input type="button" value="Print" onclick="window.print()" />
</form>
</body>
```

Output



Although it serves the purpose of getting a printout, it is not a recommended way. A printer friendly page is really just a page with text, no images, graphics, or advertising.

You can make a page printer friendly in the following ways:

1. Make a copy of the page and leave out unwanted text and graphics, then link to that printer friendly page from the original. Check Example.

2. If you do not want to keep an extra copy of a page, then you can mark your printable text using proper comments like <!-- PRINT STARTS HERE -->..... <!-- PRINT ENDS HERE --> and then you can use PERL or any other script in the background to purge printable text and display for final printing. We

tutorialspoint
SIMPLYEASYLEARNING

at Tutorialspoint use this method to provide print facility to our site visitors. Check Example.

How to Print a Page?

If you don’t find the above facilities on a web page, then you can use the browser's standard toolbar to get print the web page. Follow the link as follows.

```
File --> Print --> Click OK button.
```


tutorialspoint
SIMPLYEASYLEARNING

WWW.ROLASOFTTECH.COM

tutorialspoint
SIMPLYEASYLEARNING